

🔗 Spring Microservices V3 (in28minutes) — Interview Study Guide

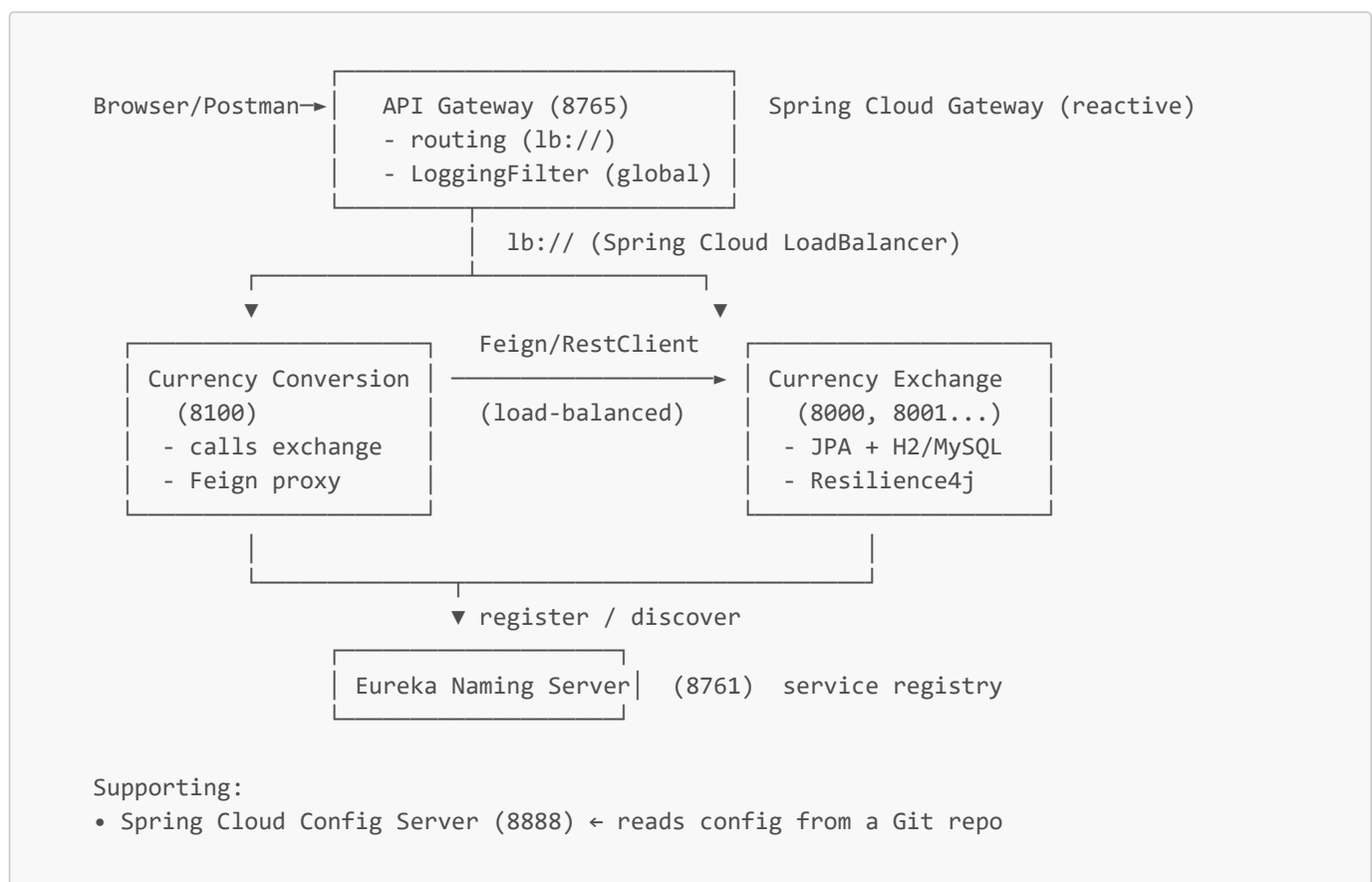
Built from a deep code walk-through of the [spring-microservices-v3](#) repo (in28minutes / Ranga Karanam). The course has **two halves**: (1) build proper **RESTful web services** with Spring Boot, then (2) turn them into **Spring Cloud microservices**, and finally **containerize** (Docker) & **orchestrate** (Kubernetes).

0. The Big Picture (memorize this first)

The course is organized into folders that are **learning modules**, not one app:

Folder	What it teaches
02.restful-web-services	REST API fundamentals (one Spring Boot app: users, posts, hello-world, versioning, filtering, validation, HATEOAS, i18n, security, JPA)
03.microservices	6 Spring Cloud services (the currency example)
04.docker	Containerize the 4 microservices + Zipkin via docker-compose
05.kubernetes	Deploy currency services to GKE with Deployments/Services/ConfigMaps/Probes/HPA
97.new-features	Lombok, JDK features, docker-compose notes
98.microservices-starting-point...	starter code for V1 learners
99.presentation, prep	slides / prep

The Microservices architecture (currency example)



- Limits Service (8080) ← demo client of Config Server / @ConfigurationProperties
- Zipkin (9411) ← distributed tracing (Micrometer → Brave → Zipkin)

Service	Port	Role
Limits Service	8080	Demo of centralized config + @ConfigurationProperties
Spring Cloud Config Server	8888	Serves config from Git to all services
Currency Exchange	8000	Provider: returns exchange rate (JPA+H2)
Currency Conversion	8100	Consumer: calls Exchange via Feign, multiplies by quantity
Naming Server (Eureka)	8761	Service registry / discovery
API Gateway	8765	Single entry point, routing, cross-cutting concerns
Zipkin	9411	Distributed tracing UI

PART 1 — RESTful Web Services (02.restful-web-services)

One Spring Boot app demonstrating every REST concept. Package:
`com.in28minutes.rest.webservices.restfulwebservices.`

1.1 What is REST? (say this crisply)

- **REST** = REpresentational State Transfer — an architectural style using **HTTP** as the protocol.
- Key ideas: **resources** (nouns) identified by **URIs**; act on them with **HTTP methods** (verbs); exchange **representations** (JSON/XML); be **stateless**; use **HTTP status codes** and **headers** properly; **HATEOAS** for discoverability.
- **Richardson Maturity Model** (Level 0→3): 0 = HTTP as tunnel, 1 = resources, 2 = HTTP verbs + status codes, 3 = HATEOAS.
- **HTTP method** → **operation mapping**: **GET** (read), **POST** (create), **PUT** (replace/update), **DELETE** (remove), **PATCH** (partial).

1.2 First REST controller — `helloworld/HelloWorldController.java`

```
@RestController
public class HelloWorldController {
    @GetMapping("/hello-world")           String helloWorld(){ return "Hello World"; }
    @GetMapping("/hello-world-bean")      HelloWorldBean helloWorldBean(){...}    //
    returns object → JSON
    @GetMapping("/hello-world/path-variable/{name}")
    HelloWorldBean withPath(@PathVariable String name){...}                // path
    variable
}
```

- `@RestController` = `@Controller` + `@ResponseBody` → return value is serialized (Jackson) to JSON.
- `@GetMapping` is shorthand for `@RequestMapping(method=GET)`.
- **Internationalization (i18n)** — `helloWorldInternationalized()` uses `MessageSource` + `LocaleContextHolder.getLocale()` to return messages based on the `Accept-Language` header (`messages.properties`, `messages_nl.properties`..).

1.3 REST CRUD + best practices — `user/UserResource.java`

```

@GetMapping("/users")                List<User> retrieveAllUsers(){...}
@GetMapping("/users/{id}")           EntityModel<User> retrieveUser(@PathVariable int id){...}
@PostMapping("/users")              ResponseEntity<User> createUser(@Valid @RequestBody User
user){
    User saved = service.save(user);
    URI location = ServletUriComponentsBuilder.fromCurrentRequest()
        .path("/{id}").buildAndExpand(saved.getId()).toUri();
    return ResponseEntity.created(location).build();    // 201 + Location header
}
@DeleteMapping("/users/{id}")        void deleteUser(@PathVariable int id){...}

```

Talking points the course hammers:

- Return **proper status codes**: 200 OK, 201 Created (+ Location header pointing to the new resource), 404 Not Found, 400 Bad Request.
- Use **ResponseEntity** to control status + headers.
- **@RequestBody** binds JSON → object; **@PathVariable** binds URI segments.

1.4 Exception handling — `exception/`

- Custom exception `UserNotFoundException` annotated `@ResponseStatus(HttpStatus.NOT_FOUND)`.
- **Global handler** `CustomizedResponseEntityExceptionHandler` extends `ResponseEntityExceptionHandler`:

```

@RestControllerAdvice
public class CustomizedResponseEntityExceptionHandler extends ResponseEntityExceptionHandler
{
    @ExceptionHandler(Exception.class)           // catch-all → 500 + ErrorDetails
    @ExceptionHandler(UserNotFoundException.class) // → 404
    @Override handleMethodArgumentNotValid(...)    // validation errors → 400 + first error
}

```

- `ErrorDetails` = consistent error body `{timestamp, message, details}`.
- **Q: Why a global exception handler?** Consistent error structure across the API, separated from business logic.

1.5 Validation — `user/User.java`

```

@Size(min=2, message="Name should have atleast 2 characters") private String name;
@Past(message="Birth Date should be in the past")              private LocalDate birthDate;

```

- Triggered by **@Valid** on **@RequestBody**. Failures → `MethodArgumentNotValidException` → handled above → 400.
- Uses Jakarta Bean Validation (`spring-boot-starter-validation`).

1.6 HATEOAS — `user/UserResource.retrieveUser`

```

EntityModel<User> entityModel = EntityModel.of(user);
WebMvcLinkBuilder link = linkTo(methodOn(this.getClass()).retrieveAllUsers());
entityModel.add(link.withRel("all-users"));
return entityModel;    // response includes "_links": { "all-users": {...} }

```

- **HATEOAS** = Hypermedia As The Engine Of Application State → responses carry **links** to related actions, so clients don't hardcode URIs.
- Spring HATEOAS: `EntityModel` (wraps data + links), `WebMvcLinkBuilder` (`linkTo`, `methodOn`).

1.7 API Versioning — `versioning/VersioningPersonController.java`

Four strategies, all demoed:

Strategy	Example	Pros/Cons
URI versioning	<code>/v1/person</code> , <code>/v2/person</code>	Simple, very common; pollutes URI space
Request param	<code>/person?version=1</code>	Easy; messy URIs
(Custom) Header	<code>X-API-VERSION=1</code>	Clean URIs; harder to test in browser
Media type / "Content negotiation"	<code>Accept: application/vnd.company.app-v1+json</code>	"Correct" REST; complex

- **Factors when choosing:** URI pollution, misuse of HTTP headers, caching (header-based breaks URL caching), browser testability, API documentation support. **No perfect option** — be consistent.

1.8 Content Negotiation & XML

- Same URL can return **JSON or XML** based on the `Accept` header (`application/json` vs `application/xml`).
- Add `jackson-dataformat-xml` for XML; course recommends **disabling XML** so browsers default to JSON.

1.9 Response Filtering — `filtering/`

- **Static filtering** (same for all responses): `@JsonIgnore` on a field, or `@JsonIgnoreProperties({"field1"})` on the class. Also `@JsonProperty("user_name")` to rename.
- **Dynamic filtering** (different fields per endpoint): `@JsonView` (in `SomeBean` + `View` interfaces) or `@JsonFilter` + `MappingJacksonValue`.

```
class SomeBean {
    @JsonView(View.View1.class) String field1;
    @JsonView(View.View2.class) String field2;
    @JsonView({View.View1.class, View.View2.class}) String field3;
}
@GetMapping("/filtering-with-view") @JsonView(View.View1.class) // returns field1 + field3
```

1.10 Connecting to a DB with JPA — `user/UserJpaResource.java`, `jpa/`*

- `@Entity(name="user_details")` User with `@Id` `@GeneratedValue`, `@OneToMany(mappedBy="user")` `List<Post>`, `@JsonIgnore` to avoid infinite recursion.
- `Post` has `@ManyToOne` back to User.
- **Repositories:** `UserRepository` extends `JpaRepository<User, Integer>`, `PostRepository` extends `JpaRepository<Post, Integer>` → free CRUD (`findAll`, `findById`, `save`, `deleteById`).
- `UserJpaResource` mirrors `UserResource` but persists via repositories; adds `/jpa/users/{id}/posts` (nested resource) to read & create posts for a user.

- **H2** in-memory DB for dev (`spring.datasource.url=jdbc:h2:mem:...`, `data.sql` seeds rows, `spring.jpa.defer-datasource-initialization=true`). Swap to **MySQL** by changing datasource + dependency.
- `UserDaoService` (`@Component`) is the **pre-JPA** static-list version to contrast with JPA.

1.11 API Documentation — Swagger / OpenAPI

- `springdoc-openapi` auto-generates the **OpenAPI 3** spec at `/v3/api-docs` and **Swagger UI** at `/swagger-ui/index.html`. (Older course versions used SpringFox.)

1.12 Actuator & HAL Explorer

- **Spring Boot Actuator** exposes ops endpoints (`/actuator/health`, `/metrics`, `/beans`, `/mappings...`). Enable via `management.endpoints.web.exposure.include=*`.
- **HAL Explorer** (`spring-data-rest-hal-explorer`) — a UI to browse HATEOAS APIs.

1.13 Security — `security/SpringSecurityConfiguration.java`

- Add `spring-boot-starter-security` → **everything is secured with HTTP Basic** + a generated password by default.
- The commented `SecurityFilterChain` bean shows the modern (Spring Security 6 / Boot 3) lambda DSL:

```
http.authorizeHttpRequests(auth -> auth.anyRequest().authenticated());
http.httpBasic(withDefaults());
http.csrf(csrf -> csrf.disable());    // disabled so POST/PUT work without CSRF token
return http.build();
```

- **CSRF** matters for browser POST/PUT; disabled here because REST clients send credentials each time (stateless).

Part 1 likely Qs: REST principles & Richardson model? Status codes for create/not-found/validation? `@RestController` vs `@Controller`? How does `@Valid` work and where do errors surface? HATEOAS — what & why? 4 versioning strategies & trade-offs? Static vs dynamic filtering? Content negotiation? How does Spring Data JPA give you CRUD for free?

PART 2 — Spring Cloud Microservices (`03.microservices`)

2.1 Why microservices & the challenges

- **Microservices** = small, autonomous, independently deployable services, each owning a capability + its data.
- **New challenges** that Spring Cloud solves (map each challenge → tool):

Challenge	Spring Cloud solution (in this repo)
Centralized configuration	Spring Cloud Config Server
Dynamic service location	Eureka Naming Server (discovery)
Load balancing	Spring Cloud LoadBalancer (replaces Ribbon)
Easy REST clients	OpenFeign
Resilience / fault tolerance	Resilience4j (replaces Hystrix)
Edge concerns / routing	Spring Cloud Gateway (replaces Zuul)

Challenge	Spring Cloud solution (in this repo)
Visibility / tracing	Micrometer + Zipkin

- **Version note this course stresses:** Ribbon→Spring Cloud LoadBalancer, Hystrix→Resilience4j, Zuul→Spring Cloud Gateway, Sleuth→Micrometer Tracing.

2.2 Centralized Configuration — Config Server + Limits Service

Config Server — `spring-cloud-config-server`:

```
@EnableConfigServer @SpringBootApplication
public class SpringCloudConfigServerApplication {...} // port 8888
```

`application.properties`:

```
spring.cloud.config.server.git.uri=file:///.../git-localconfig-repo
```

- Reads config from a **Git repo** (`limits-service.properties`, `limits-service-dev.properties`, `limits-service-qa.properties`).
- Served at `http://localhost:8888/{application}/{profile}` e.g. `/limits-service/default`.

Limits Service (client) — `limits-service`:

```
@Component @ConfigurationProperties("limits-service")
public class Configuration { private int minimum; private int maximum; ... }
```

```
spring.config.import=optional:configserver:http://localhost:8888
spring.profiles.active=qa
spring.cloud.config.profile=qa
```

- `@ConfigurationProperties("limits-service")` binds `limits-service.minimum/maximum` from config into a bean → `LimitsController` returns them.
- **Profiles** (`dev/qa/prod`) let one codebase pull environment-specific config.
`spring.config.import=configserver:` is how SB3+ clients connect (replaced `bootstrap.properties`).

Qs: Why a config server? How do profiles work? `@Value` vs `@ConfigurationProperties`? Where are secrets/config stored (Git) and how is per-env config served?

2.3 Currency Exchange (Provider) — `currency-exchange-service` (8000)

- `CurrencyExchange` `@Entity` (`id`, `from`, `to`, `conversionMultiple`, `environment`).
- `CurrencyExchangeRepository` extends `JpaRepository` with a derived query `findByFromAndTo(from, to)`.
- `CurrencyExchangeController`:

```
@GetMapping("/currency-exchange/from/{from}/to/{to}")
public CurrencyExchange retrieve(@PathVariable String from, @PathVariable String to){
    CurrencyExchange ce = repository.findByFromAndTo(from, to);
    String port = environment.getProperty("local.server.port"); // prove which instance
```

```
answered
    ce.setEnvironment(port);
    return ce;
}
```

- **Dynamic port in the response** (`local.server.port`) is a teaching trick: when you run multiple instances and load-balance, you can see which instance served the request.

2.4 Currency Conversion (Consumer) — `currency-conversion-service` (8100)

Calls Exchange **two ways**:

(a) RestClient / RestTemplate (manual HTTP):

```
CurrencyConversion cc = restClient.get()
    .uri("http://localhost:8000/currency-exchange/from/{from}/to/{to}", uriVariables)
    .retrieve().body(CurrencyConversion.class);
```

- `RestTemplate` (older) → `RestClient` (Spring 6.1+, fluent). Problem: **URL/port is hardcoded** and there's lots of boilerplate.

(b) OpenFeign (declarative) — `CurrencyExchangeProxy.java`:

```
@FeignClient(name="currency-exchange")           // resolved via Eureka, load-balanced
public interface CurrencyExchangeProxy {
    @GetMapping("/currency-exchange/from/{from}/to/{to}")
    CurrencyConversion retrieveExchangeValue(@PathVariable String from, @PathVariable String
to);
}
```

- Enabled by `@EnableFeignClients` on the main class.
- **Feign removes boilerplate**: you declare an interface, Feign generates the HTTP client, and `name="currency-exchange"` is **resolved through Eureka** (no hardcoded host/port) + **load-balanced** automatically.
- `@FeignClient(name=..., url="localhost:8000")` would pin a URL; dropping `url` makes it use discovery.

Q: RestTemplate vs Feign? Feign is declarative (interface only), integrates with Eureka + LoadBalancer + Resilience4j out of the box; RestTemplate/RestClient are imperative and need manual URLs/load-balancing.

2.5 Service Discovery — Eureka Naming Server (8761)

```
@EnableEurekaServer @SpringBootApplication
public class NamingServerApplication {...}
```

Every service is a **Eureka client**:

```
eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka
spring.application.name=currency-exchange # the name others use to find it
```

- **Flow:** services **register** with Eureka (name + host/port) and send **heartbeats**; consumers **fetch the registry** and call by **name**.
- **Why?** In the cloud, instances/ports are dynamic; hardcoding URLs breaks. Discovery decouples *where* a service is from *who* calls it.

2.6 Load Balancing — Spring Cloud LoadBalancer

- When you run multiple Exchange instances (`--server.port=8001`, etc.), Feign + **Spring Cloud LoadBalancer** distribute calls across the registered instances (client-side, round-robin by default).
- **Ribbon is deprecated** → Spring Cloud LoadBalancer is the replacement (course explicitly notes this).
- You see it work via the `environment/port` field in the response rotating.

2.7 API Gateway — `api-gateway` (8765)

Spring Cloud Gateway (reactive, built on WebFlux/Netty). Single entry point for all microservices.

Explicit routes — `ApiGatewayConfiguration.java`:

```
@Bean RouteLocator gatewayRouter(RouteLocatorBuilder b){
    return b.routes()
        .route(p -> p.path("/currency-exchange/**").uri("lb://currency-exchange"))
        .route(p -> p.path("/currency-conversion/**").uri("lb://currency-conversion"))
        .route(p -> p.path("/currency-conversion-feign/**").uri("lb://currency-conversion"))
        .route(p -> p.path("/currency-conversion-new/**")
            .filters(f -> f.rewritePath("/currency-conversion-new/(?<segment>.*)",
                "/currency-conversion-feign/${segment}"))
            .uri("lb://currency-conversion"))
        .route(p -> p.path("/get").filters(f -> f.addRequestHeader("MyHeader", "MyURI")
            .addRequestParameter("Param", "MyValue"))
            .uri("http://httpbin.org:80"))
        .build();
}
```

- `lb://currency-exchange` = route to a **discovered, load-balanced** service by name.
- **Predicates** (path) choose the route; **Filters** mutate request/response (`rewritePath`, `addRequestHeader`).
- **Discovery Locator** mode (commented in `application.properties`):
`spring.cloud.gateway.discovery.locator.enabled=true` auto-creates routes for every Eureka service (no manual route config).

Global filter (cross-cutting) — `LoggingFilter.java`:

```
@Component
public class LoggingFilter implements GlobalFilter {
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain){
        logger.info("Path of the request received -> {}", exchange.getRequest().getPath());
        return chain.filter(exchange);
    }
}
```

- `GlobalFilter` runs for **every** request → ideal for logging, auth, metrics.

Qs: What does an API gateway do? Predicate vs filter? `lb://` meaning? Discovery-locator vs explicit routes? `GlobalFilter` use cases? Why reactive (Netty)?

2.8 Fault Tolerance — Resilience4j — `CircuitBreakerController.java`

Four patterns, each demoed by swapping the annotation:

```
@GetMapping("/sample-api")
//@Retry(name="sample-api", fallbackMethod="hardcodedResponse")
//@CircuitBreaker(name="default", fallbackMethod="hardcodedResponse")
//@RateLimiter(name="default")
@Bulkhead(name="sample-api")
public String sampleApi(){ ... }
public String hardcodedResponse(Exception ex){ return "fallback-response"; }
```

Annotation	Purpose	Config keys (<code>application.properties</code>)
<code>@Retry</code>	Re-attempt failed calls, optional exponential backoff	<code>resilience4j.retry.instances.sample-api.maxRetryAttempts=5, waitDuration=1s, enableExponentialBackoff=true</code>
<code>@CircuitBreaker</code>	Stop calling a failing service; fail fast; fallback	<code>resilience4j.circuitbreaker.instances.default.failureRateThreshold=90</code>
<code>@RateLimiter</code>	Cap #calls per time window	<code>ratelimiter.instances.default.limitForPeriod=2, limitRefreshPeriod=10s</code>
<code>@Bulkhead</code>	Limit concurrent calls (isolate resources)	<code>bulkhead.instances.default.maxConcurrentCalls=10</code>

- **fallbackMethod** returns a safe default when the call fails / breaker is open. Signature must match + take an `Exception/Throwable`.
- **Circuit Breaker states:** CLOSED → (failures exceed threshold) → OPEN (fail fast) → HALF_OPEN (trial calls) → CLOSED/OPEN.
- **Hystrix is dead** → **Resilience4j** is the modern, lightweight, functional replacement.
- Requires `resilience4j-spring-boot3` + `spring-boot-starter-aop` (Boot 4: `spring-boot-starter-aspectj`).

Qs: Difference between the 4 patterns? Retry vs Circuit Breaker? CB states? What is a fallback? Why Resilience4j over Hystrix?

2.9 Distributed Tracing — Micrometer + Zipkin

- **Problem:** one request hops Gateway → Conversion → Exchange. How do you trace it end-to-end?
- **Solution: Micrometer Tracing** (Brave bridge) assigns a **traceld** (whole request) + **spanId** (each hop) and reports spans to **Zipkin**.
- Config (every service):

```
management.tracing.sampling.probability=1.0 # trace 100% (dev)
logging.pattern.level=%5p [${spring.application.name:},%X{traceId:-},%X{spanId:-}]
```

- `traceId`/`spanId` show up in **logs** and the **Zipkin UI** (9411), so you correlate logs↔traces.
- **History:** `spring-cloud-sleuth` → **Micrometer Tracing** (Boot 3+); dependencies `micrometer-tracing-bridge-brave` + `zipkin-reporter-brave` (or `spring-boot-starter-zipkin` in Boot 4).
- Run Zipkin: `docker run -p 9411:9411 openzipkin/zipkin`.

Qs: `traceId` vs `spanId`? How is a request traced across services? What replaced Spring Cloud Sleuth?

PART 3 — Docker (04.docker)

3.1 Docker fundamentals (vocabulary they test)

- **Image** = read-only template (app + deps + runtime). **Container** = a running instance of an image.
- **Registry** (Docker Hub) hosts **repositories**; an image is identified by `repository:tag` (e.g., `in28min/mmv3-currency-exchange-service:0.0.1-SNAPSHOT`).
- **Docker architecture:** Docker **Client** (CLI) → **Docker Daemon/Engine** → images/containers; pulls from registry.
- **Why Docker?** Same image runs identically on any machine/OS/cloud → **portability**, standard deployment, isolation, density (lighter than VMs — shares the host kernel).
- Key commands: `docker build`, `docker run -p 8000:8000 img`, `docker images`, `docker ps [-a]`, `docker logs -f`, `docker stop/rm`, `docker push/pull`, `docker stats`, `docker system prune`.

3.2 Building images without a Dockerfile — Buildpacks

`pom.xml` (spring-boot-maven-plugin):

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image><name>in28min/mmv3-${project.artifactId}:${project.version}</name></image>
    <pullPolicy>IF_NOT_PRESENT</pullPolicy>
  </configuration>
</plugin>
```

- `mvn spring-boot:build-image` uses **Cloud Native Buildpacks** to produce an optimized image — **no Dockerfile needed**. (Google **Jib** is the other no-Dockerfile option mentioned across in28minutes courses.)

3.3 Orchestrating the stack — docker-compose

`docker-compose-05-zipkin.yaml` runs the whole system:

```
services:
  currency-exchange:
    image: in28min/mmv3-currency-exchange-service:0.0.1-SNAPSHOT
    ports: ["8000:8000"]
    networks: [currency-network]
    depends_on: [naming-server]
    environment:
```

```

EUREKA.CLIENT.SERVICEURL.DEFAULTZONE: http://naming-server:8761/eureka
MANAGEMENT.ZIPKIN.TRACING.ENDPOINT: http://zipkin-server:9411/api/v2/spans
currency-conversion: { ... }
api-gateway: { ... }
naming-server: { image: ...mmv3-naming-server, ports: ["8761:8761"] }
zipkin-server:
  { image: openzipkin/zipkin:2.23, ports: ["9411:9411"], restart: always }
networks:
  currency-network:

```

- **One command** (`docker-compose up`) starts everything on a shared **bridge network** (`currency-network`) where services reach each other **by service name** (`http://naming-server:8761`).
- `depends_on` controls start order; **environment overrides** Spring properties (note the env-var form `EUREKA.CLIENT...` maps to `eureka.client...`); `mem_limit` caps memory.
- **Why service names work:** Docker Compose provides DNS for the user-defined network.

Qs: Image vs container vs registry/repository/tag? Why Docker? Buildpacks vs Dockerfile? What does docker-compose give you over `docker run`? How do containers find each other?

PART 4 — Kubernetes (05.kubernetes)

4.1 Why Kubernetes

- Docker **runs** containers; **Kubernetes orchestrates** them across a **cluster**: scheduling, **self-healing**, **auto-scaling** (HPA), **rolling updates/rollbacks**, **service discovery + load balancing**, config/secret management.
- The course deploys to **GKE (Google Kubernetes Engine)**.

4.2 Core objects (the "PRSD" mental model)

- **Pod** — smallest deployable unit; wraps 1+ containers (share network/storage). Ephemeral.
- **ReplicaSet** — keeps **N** identical pod replicas alive.
- **Deployment** — manages ReplicaSets → enables **rolling updates & rollbacks**.
- **Service** — stable virtual IP + DNS in front of pods (since pods are ephemeral). Types: **ClusterIP** (internal), **NodePort**, **LoadBalancer** (external).

4.3 Declarative YAML — `currency-exchange .../deployment-04-final.yaml`

```

apiVersion: apps/v1
kind: Deployment
metadata: { name: currency-exchange, labels: { app: currency-exchange } }
spec:
  replicas: 1
  selector: { matchLabels: { app: currency-exchange } }
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxSurge: 25%, maxUnavailable: 25% } # zero-downtime upgrades
  template:
    metadata: { labels: { app: currency-exchange } }
    spec:
      containers:
        - image: in28min/mmv3-currency-exchange-service:0.0.11-SNAPSHOT
          readinessProbe:
            { httpGet: { port: 8000, path: /actuator/health/readiness } }

```

```

    livenessProbe:
      { httpGet: { port: 8000, path: /actuator/health/liveness } }
---
apiVersion: v1
kind: Service
metadata: { name: currency-exchange }
spec:
  selector: { app: currency-exchange }
  ports: [{ port: 8000, targetPort: 8000 }]
  type: LoadBalancer

```

- **Declarative** = you describe *desired state*; K8s **reconciles** actual→desired (controllers recreate failed pods). Apply with `kubectl apply -f`.
- **Rolling update** (`maxSurge/maxUnavailable`) = update pods gradually → **no downtime**; `kubectl rollout undo` rolls back.

4.4 Health Probes (link back to Actuator)

- **livenessProbe** → "is the app alive? if not, **restart** the pod." (`/actuator/health/liveness`)
- **readinessProbe** → "is the app ready for traffic? if not, **don't route** to it." (`/actuator/health/readiness`)
- Spring Boot Actuator exposes these K8s-style probes when running in K8s (or via `management.health.probes.enabled=true`).

4.5 Service-to-service config — ConfigMaps — `currency-conversion deployment-03-final.yaml`

```

spec:
  containers:
    - image: in28min/mmv3-currency-conversion-service:0.0.12-SNAPSHOT
      envFrom:
        - configMapRef: { name: currency-conversion }
---
apiVersion: v1
kind: ConfigMap
metadata: { name: currency-conversion }
data:
  CURRENCY_EXCHANGE_URI: http://currency-exchange # K8s DNS name of the exchange Service

```

- **ConfigMap** externalizes config (12-factor) and is injected as **env vars** (`envFrom`).
- Inside the cluster, Conversion reaches Exchange at `http://currency-exchange` — **Kubernetes' own DNS/Service discovery** (so you can drop Eureka on K8s!). This is **server-side discovery** vs Eureka's client-side.

4.6 Scaling & ops commands

- `kubectl create deployment ...`, `kubectl expose deployment ... --type=LoadBalancer --port=8000`
- `kubectl apply -f deployment.yaml`, `kubectl get pods/services/deployments/all`, `kubectl describe pod <id>`
- `kubectl scale deployment currency-exchange --replicas=3`
- **Autoscaling (HPA)**: `kubectl autoscale deployment currency-exchange --min=1 --max=3 --cpu-percent=70`
- `kubectl set image deployment/... =img:tag` (rolling update), `kubectl rollout history/undo ...`

Qs: Pod vs ReplicaSet vs Deployment vs Service? ClusterIP vs NodePort vs LoadBalancer? Liveness vs readiness probe? How does K8s self-heal? Rolling update & rollback? ConfigMap vs Secret? How do services discover each other in K8s (DNS) — and why might you drop Eureka?

5. ⚡ Rapid-fire interview Q&A

REST

- *REST principles?* Resources+URIs, HTTP verbs, representations (JSON), statelessness, status codes, HATEOAS (Richardson L0–3).
- *Status codes?* 200 OK, 201 Created (+Location), 204 No Content, 400 Bad Request, 401/403, 404 Not Found, 500.
- *@RestController vs @Controller?* `@RestController = @Controller + @ResponseBody` (returns data, not views).
- *Validation?* `@Valid` + Bean Validation (`@Size,@Past`) → `MethodArgumentNotValidException` → `@RestControllerAdvice` → 400.
- *Versioning?* URI / param / header / media-type — trade-offs around URI pollution, caching, browser testing.
- *Filtering?* Static (`@JsonIgnore/@JsonIgnoreProperties`) vs dynamic (`@JsonView/@JsonFilter`).
- *HATEOAS?* Links in responses (`EntityModel+WebMvcLinkBuilder`) for discoverability.

Spring Cloud

- *Config Server?* Centralized, Git-backed, per-profile config; clients use `spring.config.import=configserver:.`
- *Eureka?* Client-side discovery: register + heartbeat + fetch registry; call by `spring.application.name`.
- *Feign vs RestTemplate/RestClient?* Declarative interface vs imperative; Feign integrates discovery+LB+Resilience4j.
- *Load balancing?* Spring Cloud LoadBalancer (Ribbon is deprecated), client-side round-robin.
- *Gateway?* Reactive edge; predicates+filters; `lb://name` routing; `GlobalFilter` for cross-cutting; discovery-locator.
- *Resilience4j?* Retry, CircuitBreaker (CLOSED/OPEN/HALF_OPEN), RateLimiter, Bulkhead + `fallbackMethod` (replaces Hystrix).
- *Tracing?* Micrometer Tracing + Zipkin; `traceld (request) + spanId (hop)`; replaced Sleuth.

Docker / K8s

- *Image vs container?* Template vs running instance.
- *Buildpacks?* `spring-boot:build-image` → image without a Dockerfile.
- *docker-compose?* One file/command for multi-container apps on a shared network (DNS by service name).
- *K8s objects?* Pod < ReplicaSet < Deployment; Service for stable networking.
- *Self-healing?* Reconciliation loop: controllers drive actual→desired state.
- *Probes?* Liveness = restart; Readiness = route traffic.
- *K8s discovery?* Service DNS (server-side) — can replace Eureka in-cluster.

Version-migration trivia (this course loves these)

- Ribbon → **Spring Cloud LoadBalancer**
- Hystrix → **Resilience4j**
- Zuul → **Spring Cloud Gateway**
- Spring Cloud Sleuth → **Micrometer Tracing**
- `bootstrap.properties` → `spring.config.import`
- `RestTemplate` → **RestClient**
- SpringFox → **springdoc-openapi**

6. Suggested revision path (1 hour)

1. Re-read **§0** (architecture + port table) and the **§5 rapid-fire**.

2. REST: open `UserResource` (HATEOAS, `ResponseEntity`), `CustomizedResponseEntityExceptionHandler`, `VersioningPersonController`, `FilteringController`.
3. Microservices: `CurrencyExchangeProxy` (Feign), `CurrencyConversionController` (RestClient vs Feign), `ApiGatewayConfiguration` (routes), `CircuitBreakerController` (Resilience4j), `NamingServerApplication`, config-server + limits-service.
4. Docker: `docker-compose-05-zipkin.yaml`. K8s: `deployment-04-final.yaml` + the ConfigMap deployment.
5. Drill the **version-migration trivia** — easy interview points.

Tip: When asked "show me code," name the concrete class — e.g. "*the Feign client is `CurrencyExchangeProxy` annotated `@FeignClient(name=\"currency-exchange\")`, and the circuit breaker + fallback live in `CircuitBreakerController.sampleApi/hardcodedResponse`.*" Concrete references prove you actually built it.